Distributed Formation Control System

Fariha Al Ferdous[1]

[1]**ID: 238888**

Course: Distributed Robot Perception

Abstract:
This project aims at the implementation of Multiple Diff Drive robots sharing their state in a global formation (anything like a square or a triangle etc.) and manage to keep that global form while moving in an xy plane. What shape they will take depends on the number of robots engaged into the system. This project uses Distributed Extended Kalman Filter for Estimation of state and velocity is computed using Control Law which is then fed to the robot plant and consumed by the sensors and these measurement model is then utilized by the Kalman estimator and formation controller and this closed-loop system continues by making the shape fixed formed by the robots. For example, for a square shape, the 4 robots will move while maintaining the squared shape. The shape changes automatically given the number of robots, number that should be discovered by the team autonomously and in a distributed way. For this, I have used Matlab Simulink for the implementation and Simscape Multibody libraries for the integration of the robot plant bodies and all other components.

I INTRODUCTION

This project focuses on a distributed formation control problem, where multiple Diff Drive Robots move from their initial positions in order to form a shape which depends on their counts, for example, A square will get formed if the robot count is 4. Each robot has been assigned unique IDs and based on this, which corner will be taken by which robot has been decided. But the formation choice whether a straight line or triangle or square is decided automatically depending on how many robot subsystems are engaged into the system and upon this decision the robots take shape automatically. However, shape formation depends on the spacing between the robots and the positions of each of the robots and which robot will take which corner is fixed and is based on their IDs. For position determination I have used distributed extended kalman filter to share state estimates and finally reaching a consensus position per each robot then based on each consensus position a centroid is calculated and a position for each of the robots is determined based on the formation type and then robots move to take position until error goes to near zero and the motion works on PD closed loop control law.

Using the Simscape Multibody extension in Matlab, I have created the diff drive robot joining each joints and bodies as per requirement, and set initial positions for all of the robots. All these parameters including initial positions are then fed by the sensors and adding noise I made them work in the further processes. White noise has been added to make a realistic simulation. The shape formation follows the below criteria:

1. A line for 2 robots.
2. A triangle for 3 robots.
3. A square for 4 robots.

A Problem Formulation

A.1 Unicycle Kinematics (Differential Drive Plant)

Consider a group of N non-holonomic vehicles of differential drive type. For i = 1, . . . , N , the kinematic model of vehicle i is described by

Continuous model:

$$\dot{x} = v\cos(\theta)$$
$$\dot{y} = v\sin(\theta)$$
$$\dot{\theta} = \omega$$

where:

$$v = \frac{r(\omega_L + \omega_R)}{2} \quad \text{linear velocity}$$
$$\omega = \frac{r(\omega_R - \omega_L)}{L} \quad \text{angular velocity}$$

$$r = \text{wheel radius}, \quad L = \text{wheelbase}$$

Discrete Euler ($T_s = 0.01\,s$):

$$\begin{aligned} x(k+1) &= x(k) + v(k)\cos(\theta(k))\,T_s \\ y(k+1) &= y(k) + v(k)\sin(\theta(k))\,T_s \\ \theta(k+1) &= \theta(k) + \omega(k)\,T_s \end{aligned}$$

where $[x,y]^T \in \mathbb{R}^2$ represents the Cartesian coordinates of the center of mass of the i-th robot, $v \in \mathbb{R}$ is the linear velocity, $\theta \in \mathbb{R}$ is the heading angle and $\omega \in \mathbb{R}$ is the angular velocity.

II ADOPTED MODELS

A Communication Graph

A multi-robot system is modeled as a graph:

$$G = (V, E)$$

where:

$$V = \{1, 2, \ldots, N\} \quad = \text{set of robots (nodes)}$$

$$E \subseteq V \times V \quad = \text{communication links (edges)}$$

In this system, the communication graph is undirected, i.e., all edges are bidirectional, because every robot broadcasts to all its neighbours and all neighbours receive the information.

Adjacency Matrix A

$$A_{ij} = \begin{cases} 1, & \text{if robot } j \text{ is a neighbour of robot } i \\ 0, & \text{otherwise (including } i = j, \text{ no self-loops)} \end{cases}$$

Degree Matrix D

$$D_{ii} = \sum_j A_{ij} = \text{number of neighbours of robot } i$$

Graph Laplacian L

$$L = D - A$$

For example, for $N = 3$:

$$L = \begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}$$

Key properties:

$$L\mathbf{1} = 0 \quad \text{(rows sum to zero)}$$

$$\lambda_2(L) > 0 \;\Leftrightarrow\; \text{graph is connected}$$

$$\lambda_2 = \text{algebraic connectivity (Fiedler value)}$$

$$\text{Larger } \lambda_2 \to \text{faster consensus convergence}$$

B System Model

B.1 Diff Drive Robot Plant

From the Figure 1,

Actuators used:

1. One 6-DOF joint connected with one Brick solid frame to represent chassis
2. Two Revolute joints as wheels connected with Cylindrical solid frames to represent two wheels
3. According to joint primitives the degree of freedom is 6 but according to controllable dof it is 3 (x, y, θ)
4. Transform sensors connected with all joints to coordinate the local parameters in the World coordinate frame
5. Rigid Transform bodies to maintain all frame positions and translations
6. One Planar Controller to transform the velocity commands to provide forces to the robot bodies to generate required motion
7. Velocity commands vL, vR have been converted to angles to drive the wheels for rotation of the robot plant body since revolute joints dont take velocity commands directly

Ground truth parameters received from robot plants to sensors:

1. Robot position (x_i, y_i, θ_i)
2. Quaternion q_i
3. Linear velocity v_i
4. Angular velocity w_i

B.2 Sensors

Sensors used:

1. GPS: received with white noise added (x_i, y_i, v_i)
2. IMU: received with white noise added (θ_i, w_i)

C EKF Model

Distributed Extended Kalman Filter model has been used to share the states between neighbor robots.

C.1 EKF Predict

State Prediction with Measurement Model:

$$\begin{aligned}\hat{\mathbf{x}}_\mathbf{i}(\mathbf{k}+\mathbf{1}) &= f(x_i(k), u_i(k)) \\ &= x_i(k) + v_i \cos(\theta_i)\, dt \\ &= y_i(k) + v_i \sin(\theta_i)\, dt \\ &= \theta_i + w_i\, dt \\ &= v_i + a_i\, dt \\ &= bias_gyro_i\end{aligned}$$

where $[x_i, y_i]^T \in \mathbb{R}^2$ represents the Cartesian coordinates of the center of mass of the i-th robot, $v_i \in \mathbb{R}$ is the linear velocity, $\theta_i \in \mathbb{R}$ is the heading angle, $\omega_i \in \mathbb{R}$ is the angular velocity, $a_i \in \mathbb{R}$ is the linear acceleration and $bias_gyro \in \mathbb{R}$ is the bias measured from sensor readings and $dt \in \mathbb{R}$ is the discrete timestep.

Jacobian:
$A_i(k) = \frac{\partial f}{\partial x_i}$:

$$A_i(k) = \begin{bmatrix} 1 & 0 & -v\sin(\theta)\, dt & \cos(\theta)\, dt & 0 \\ 0 & 1 & v\cos(\theta)\, dt & \sin(\theta)\, dt & 0 \\ 0 & 0 & 1 & 0 & -dt \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Noise Covariance Matrix:

$$Q = I(5) * 0.01$$

Covariance prediction:

$$\hat{\mathbf{P}}_\mathbf{i}(\mathbf{k}+\mathbf{1}) = \mathbf{A}\mathbf{P}_\mathbf{i}(\mathbf{k})\mathbf{A}^\mathbf{T} + \mathbf{Q}$$

$$P_i(k) \leftarrow \hat{P}_i(k+1)$$

C.2 EKF Update

Actual Measurement from sensors:

$$\mathbf{z}_{actual} = \mathbf{h}(\mathbf{x}) = \begin{bmatrix} gps_x \\ gps_y \\ \theta_{imu} \\ v_{gps} \\ \text{(IMU } \omega \text{ + bias)} \end{bmatrix}$$

Predicted measurement:

$$\mathbf{z}_{pred} = \mathbf{h}(\bar{\mathbf{x}}) = \begin{bmatrix} \bar{x}_1 \\ \bar{x}_2 \\ \bar{x}_3 \\ \bar{x}_4 \\ \bar{x}_5 \end{bmatrix} = \begin{bmatrix} gps_x \\ gps_y \\ imu_\theta \\ v_{gps} \\ \text{(IMU } \omega \text{ + bias)} \end{bmatrix}$$

Innovation:

$$\mathbf{y} = \mathbf{z}_{actual} - \mathbf{z}_{pred}$$

Noise Variance:

$$z_{\text{detrended}} = \mathbf{z}_{actual} - \text{mean}(z, 1)$$

$$R = \text{cov}(z_{\text{detrended}})$$

Innovation covariance:

$$S = H\hat{P}_i(k+1)H^T + R$$

where, H = Jacobian of the measurement matrix If S is ill-conditioned:

$$S \leftarrow S + \varepsilon I$$

Kalman gain:

$$W = \hat{P}_i(k+1)H^T S^{-1}$$

State update:

$$\mathbf{x}_\mathbf{i}(\mathbf{k}+\mathbf{1}) = \hat{\mathbf{x}}_\mathbf{i}(\mathbf{k}) + \mathbf{W}\mathbf{y}$$

Covariance update:

$$\mathbf{P}_\mathbf{i}(\mathbf{k}+\mathbf{1}) = (\mathbf{I} - \mathbf{W}\mathbf{H})\,\hat{\mathbf{P}}_\mathbf{i}(\mathbf{k}+\mathbf{1})$$

D Measurement Model $h(x)$

Stacked measurement vector $\mathbf{z} \in \mathbb{R}^{4\times 1}$:

$$\mathbf{z} = \begin{bmatrix} gps_x \\ gps_y \\ \theta_{imu} \\ v_{gps} \\ \text{(IMU } \omega \text{ + bias)} \end{bmatrix}$$

Measurement prediction $h(\hat{\mathbf{x}})$:

$$\begin{aligned} h_1(\hat{\mathbf{x}}) &= \hat{x}_1 && \text{(GPS x)} \\ h_2(\hat{\mathbf{x}}) &= \hat{x}_2 && \text{(GPS y)} \\ h_3(\hat{\mathbf{x}}) &= \hat{x}_3 && \text{(IMU } \theta\text{)} \\ h_4(\hat{\mathbf{x}}) &= \hat{x}_4 && \text{(GPS } v\text{)} \\ h_5(\hat{\mathbf{x}}) &= \frac{\hat{x}_3 - \theta_{prev}}{T_s} + \hat{x}_5 && \text{(IMU } \omega \text{ + bias)} \end{aligned}$$

Jacobian $H \in \mathbb{R}^{4\times 5}$:

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{1}{T_s} & 0 & 1 \end{bmatrix}$$

Noise covariance:

$$R = \text{diag}\left(\sigma_{gps}^2, \sigma_{gps}^2, \sigma_{imu}^2, \sigma_{gps}^2, \sigma_{imu}^2\right)$$

E Distributed EKF: Covariance Intersection

To get a fused state for each robot Covariance Intersection Algorithm has been adopted.

$$x_{fused} = P_{fused}\left(\alpha P_{prev}^{-1} x_{prev} + (1-\alpha) P_j^{-1} x_j\right)$$

F Information Fusion

Fusion of states and covariances from neighbor robots **State update:**

$$\hat{x}_j(:,k) = \bar{x}_j + W_j\left(z_j - \bar{x}_j\right)$$

Covariance update:

$$P_j(:,:,k) = (I - W_j H_j)\, P_{j,\text{bar}}$$

G Consensus Algorithm

Consensus update of robot state from estimated robot states of neighbors

$$\hat{x}_i^{cons}(k+1) = \hat{x}_i(k) + \gamma \sum_{j \in \mathcal{N}_i} a_{ij}\left(\hat{x}_j(k) - \hat{x}_i(k)\right)$$

$$\gamma = \text{consensus gain} \quad \text{must satisfy:} \quad \gamma < \frac{1}{\Delta_{\max}}$$

$$\Delta_{\max} = \text{maximum node degree in the graph}$$

$$a_{ij} = \begin{cases} 1, & \text{if nodes } i \text{ and } j \text{ are connected} \\ 0, & \text{otherwise} \end{cases}$$

III SOLUTION

A Centroid Formation

Formed centroid using Consensus updated robot state.

$$x_c = \frac{1}{N}\sum_{i=1}^{N} \hat{x}_i^{cons}[1:2]$$

which is,

1. average of all robots' consensus-corrected positions
2. geometric centre of the formation
3. Used as the reference point for all formation offsets δ_i.

After convergence, x_c is latched to prevent drift.

B Formation Selection

Formation offsets δ_i

- **Line formation** ($N \leq 2$)**:**

$$center = \frac{N-1}{2}$$

$$\delta_i = \begin{bmatrix} (ID_i - 1 - center)\, spacing \\ 0 \end{bmatrix}$$

- **Square formation** ($N = 4$)**:**

$$half = \frac{spacing}{2}$$

$$\delta_i = \begin{bmatrix} \pm half \\ \pm half \end{bmatrix} \quad \text{(corners assigned per } ID_i\text{)}$$

- **Triangle formation** ($N = 3$)**:**

$$side = 2 \cdot spacing, \quad h = \frac{\sqrt{3}}{2}\, side$$

$$\delta_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix} \quad \text{(vertices assigned per } ID_i\text{)}$$

- **Default:**

$$\delta_i = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

C Control Law

Error Calculation

$$e_{i,\text{pos}}^{raw} = (x_c + \delta_i) - x_{fused}[1:2]$$

where, δ_i is the robot position calculated from Formation Selection phase.

PD control (world frame):

$$u = K_p e_i - K_d \dot{e}_i$$

$$v = \cos(\theta)\, u_1 + \sin(\theta)\, u_2$$

$$\omega = -\sin(\theta)\, u_1 + \cos(\theta)\, u_2$$

$$v_L = v - \frac{L}{2}\,\omega$$

$$v_R = v + \frac{L}{2}\,\omega$$

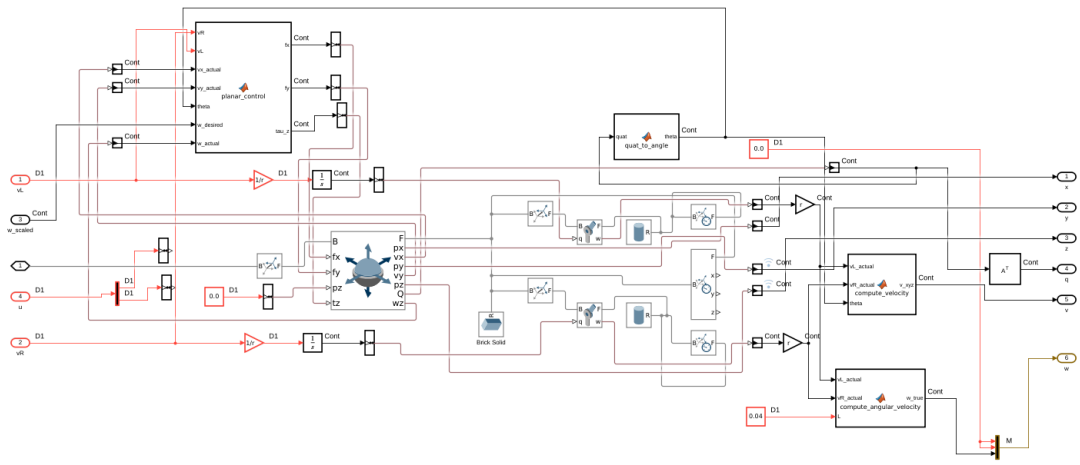

Figure 1. Robot Plant

IV IMPLEMENTATION DETAILS

A Methodology

1. Distributed Extended Kalman Filter Algorithm has been used to estimate robot states of local and neighbors both
2. These estimated robot states have been fused to a final estimate of Robot state using Covariance Intersection Algorithm
3. Using Consensus Algorithm a final consensus updated state is reached
4. This Consensus updated state is then used to calculate the centroid of the formation shape
5. Each robot calculates a centroid of their own using their own Consensus updated state
6. Each robot passes a buffer message array containing their own state, covariance, ID and timestamp.
7. These buffer messages get stored in the Data Store Memory block which is completely independent of the robot plants subsystems
8. Each message from each robot gets read by each of the robot plant and the messages get filtered using the saved timestamps in order to get the latest message for each of the robot
9. From the received message counts the actual count of the robots is calculated and broadcasted using another Data Store write blocks by each robot
10. Using a minmax block the final count of robots is gained
11. From the final count of the robots N_final, the formation shape is chosen using a switch case block
12. Using this count, the centroid, the formation shape and the fused state each robot's position is calculated using the robot ID.
13. Finally error is calculated using robot's desired position (centroid + robot's desired position) and the fused actual position
14. from this positional error, velocity error is calculated also using derivation
15. Finally, PD Control Law is used to redirect the robot with velocity commands (vL and vR)

B Materials

1. Matlab
2. Simulink
3. Simscape Multibody Library
4. Other Matlab libraries

V RESULTS

PDF of Error

As shown in Figure 2, The PDF of X Error and Y Error both is narrow which resembles a stable system and the peak of X Error and Y Error both

settle near 6-7.8 unit error and distribution is Gaussian. The distribution is dense in between 4-9 unit error.

CDF of Error

As shown in Figure 3, 10% of Error are below 4 for X Error and 5 for Y Error. Steep rise between 4.5-8.5m.

Mean, Std Deviation & 95% Confidence Interval

As shown in Figure 4, Mean = 6.5m for X Error and 7.07m for Y Error with high statistical confidence. From Table 1, Mean is 0.8083 and Overall Std Deviation is 0.5438 m. which means system does not oscillate much and giving stable performance. CI is wide with width 1.2m — reasonably tight in between 6-7.2m which resembles variance — robot actually moving around trying to converge, not stuck at fixed value.

Box Plot

As shown in Figure 5,

Bottom whisker: 3.35m (X Error) 4.0m (Y Error)

Q1 (box bottom): 6.0m (X Error) 7.7m (Y Error)

Median (orange): 6.9m (X Error) 7.5m (Y Error)

Q3 (box top): 7.50m (X Error) 7.95m (Y Error)

Top whisker: 9.80m (X Error) 10.4m(Y Error)

Outliers: Random small whiskers (Outliers)

Box centre (6.5-8.0m) = healthy steady-state range. Formation maintained within this band. There are many outlierw which resemble occasional failures.

IQR = Q3-Q1 = 7.50-6.0 = 1.50m — not that tight box, resembles outliers in data due to systemic or sensor noise.

Steady-State Error after Convergence & RMSE

As shown in Figure 6, Convergence time is at 0.111, which means Robot crossed below 0.5m threshold almost immediately.

From Table 1, Steady-State RMSE is 0.9314 m which means system needs more finetuning for better performance.

Measurement Model and Error Evolution

As shown in Figure 7,8, Z_Predicted has many spikes both upper and lower area which resemble noise in the data, same shows the Error Evolution over time plot.

Velocity over time

Figure 9 shows the evolution of Velocity over time, the reason only Vx and Vy oscilate because the robot move along x and y plane only so no motion on z axis.

A Metrics used

Metrics	Values
Convergence time	0.110 s
Steady-State RMSE	0.9314 m
Mean Error X Y	6.5241 7.0711
Overall Mean	0.8083 m
Overall Std Deviation	0.5438 m

Table 1. Metrics used

VI CONCLUSIONS

This project focused on Distributed Formation Control System where it worked on 3 types of shape formation. From the Evaluation data we can say that, the system was successful to perform and output stable result.

Github Repository: `https://github.com/farihachaiti/Distributed_Formation_Control.git`

VII LIMITATIONS

1. The system is stable but needs more tuning to filter out the outlier free data for better performance.

REFERENCES

Lin, Z., Wang, L., Han, Z., and Fu, M. (2014). Distributed formation control of multi-agent systems using complex laplacian. *IEEE Transactions on Automatic Control*, 59(7):1765–1777.

Liu, T. and Jiang, Z.-P. (2013). Distributed formation control of nonholonomic mobile robots without global position measurements. *Automatica*, 49(2):592–600.

Liu, Y., Liu, J., He, Z., Li, Z., Zhang, Q., and Ding, Z. (2024). A survey of multi-agent systems on distributed formation control. *Unmanned Systems*, 12(05):913–926.

Montijano, E., Cristofalo, E., Zhou, D., Schwager, M., and Saguees, C. (2016). Vision-based distributed formation control without an external positioning system. *IEEE Transactions on Robotics*, 32(2):339–351.

Ren, W. and Sorensen, N. (2008). Distributed coordination architecture for multi-robot formation control. *Robotics and Autonomous Systems*, 56(4):324–333.

Yu, X. and Liu, L. (2015). Distributed formation control of nonholonomic vehicles subject to velocity constraints. *IEEE Transactions on Industrial Electronics*, 63(2):1289–1298.

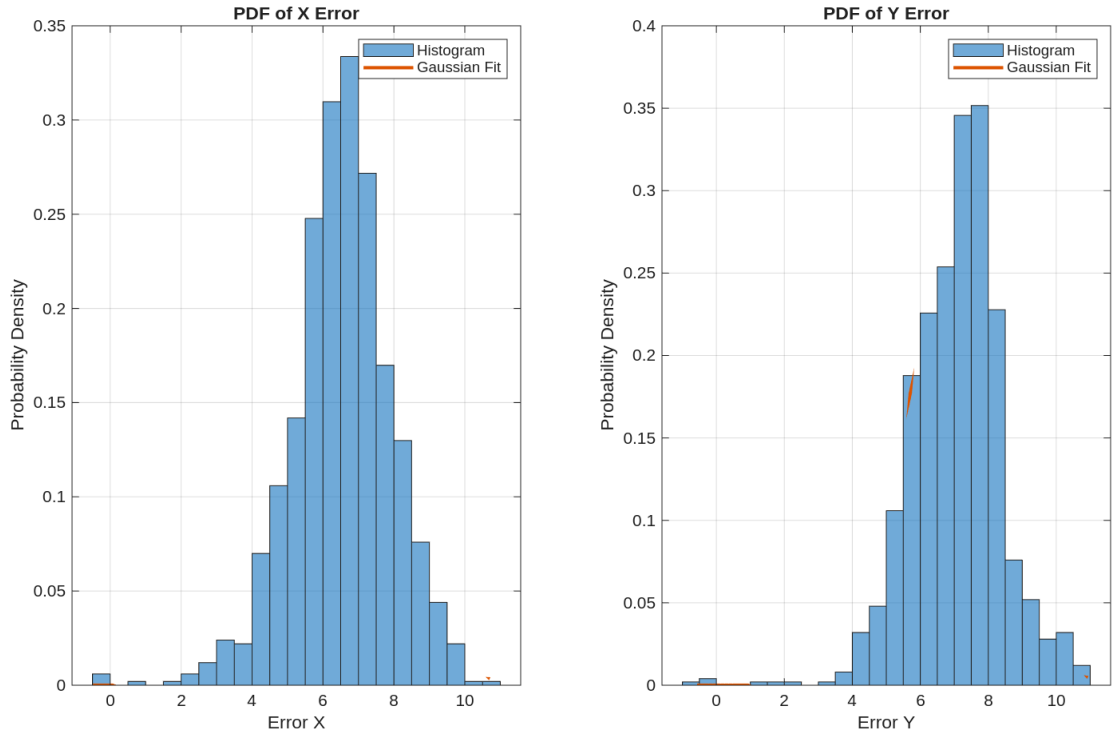

Figure 2. PDF of Position Error

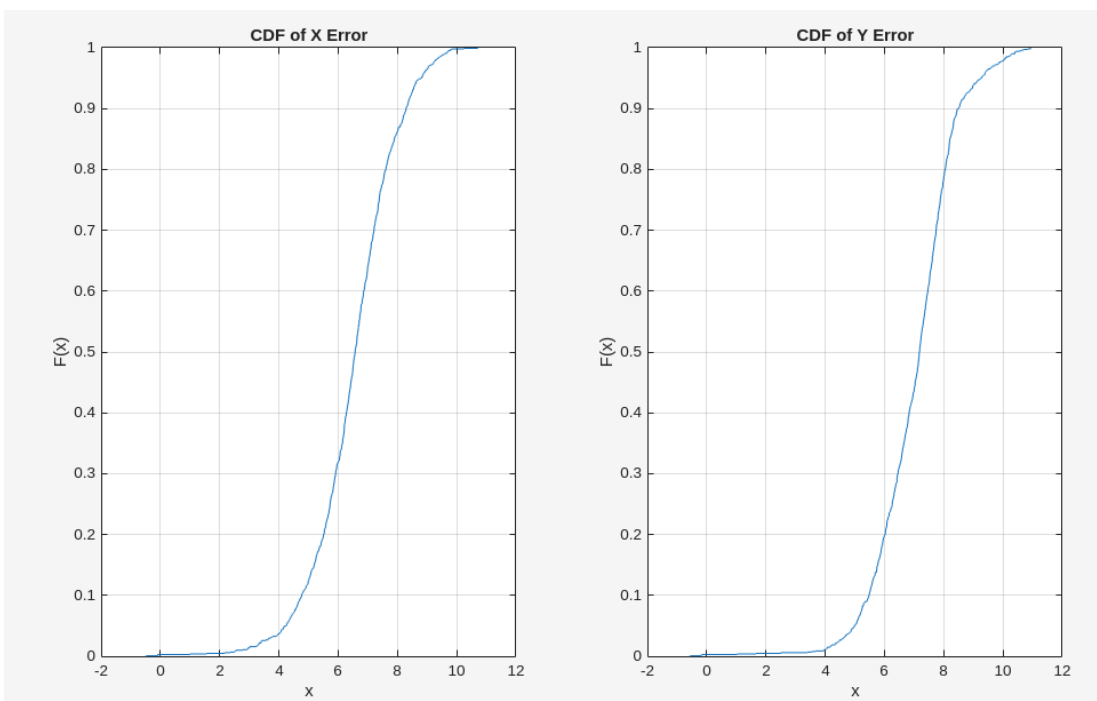

Figure 3. CDF of Position Error

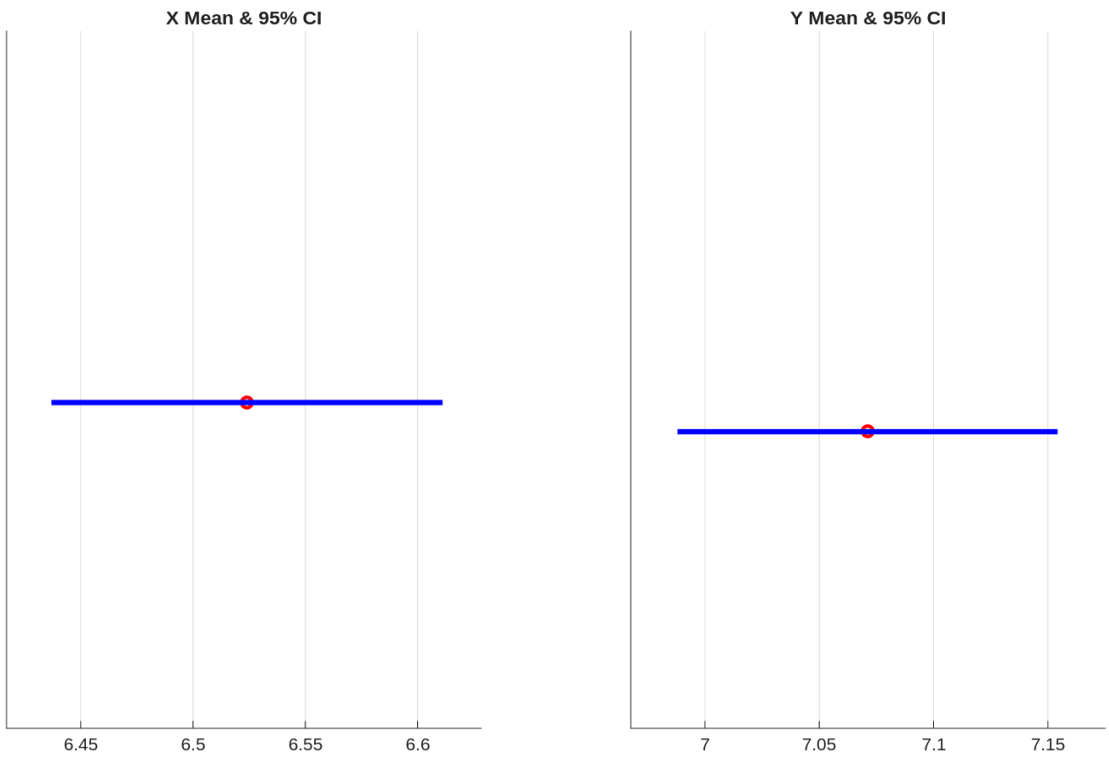

Figure 4. Mean and 95% Confidence Interval

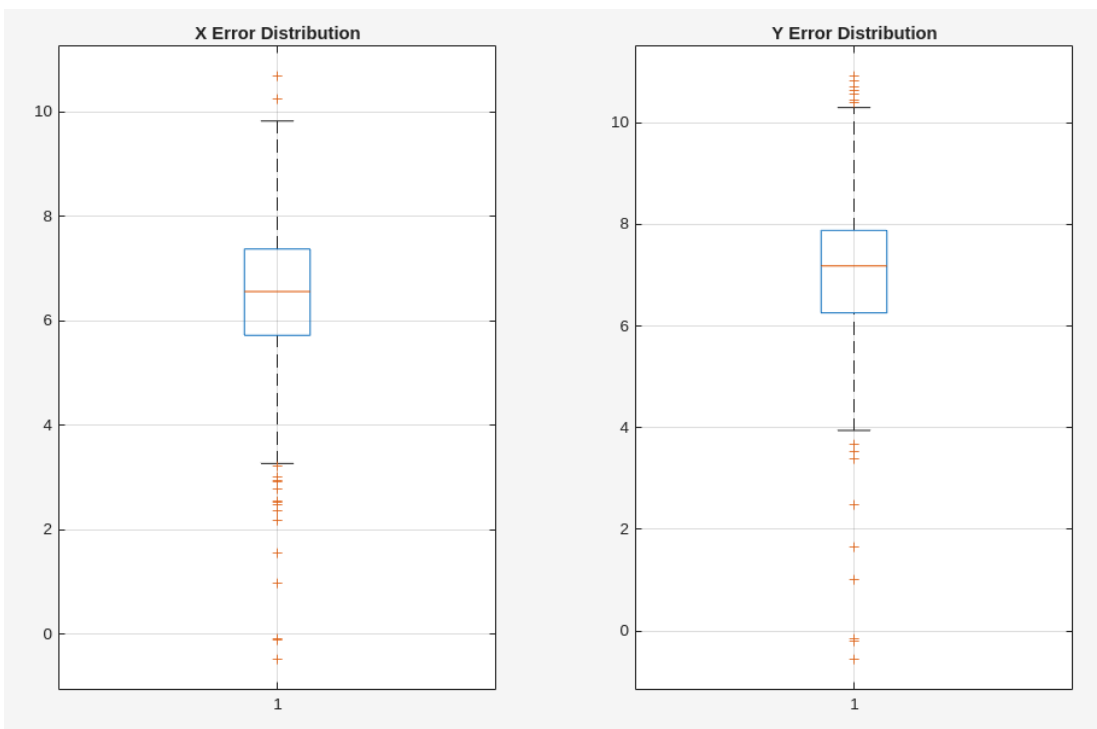

Figure 5. Error Distribution (Box Plot)

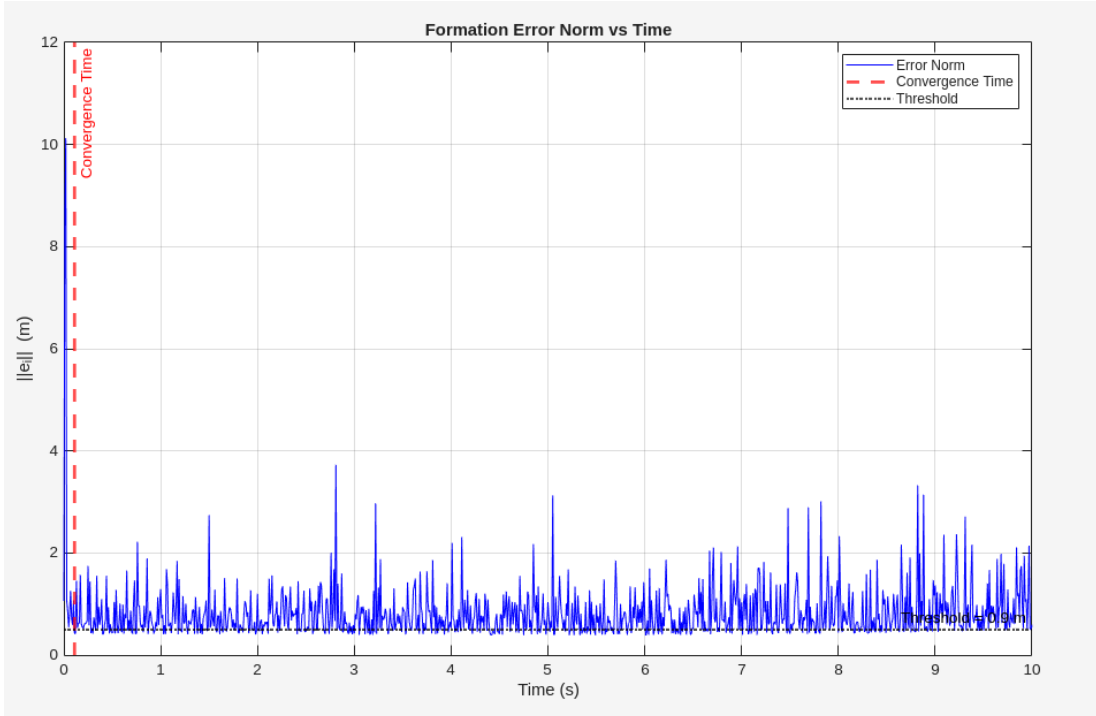

Figure 6. Steady-State Error after Convergence

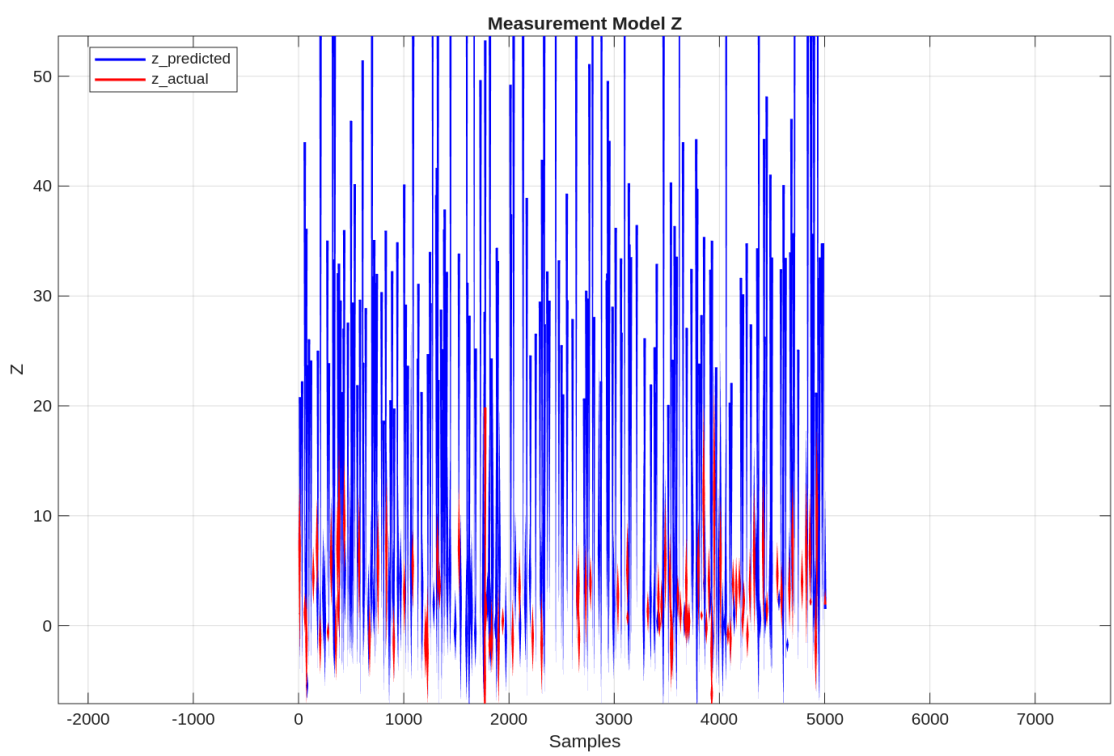

Figure 7. Measurement Model Comparison zPredicted vs zActual

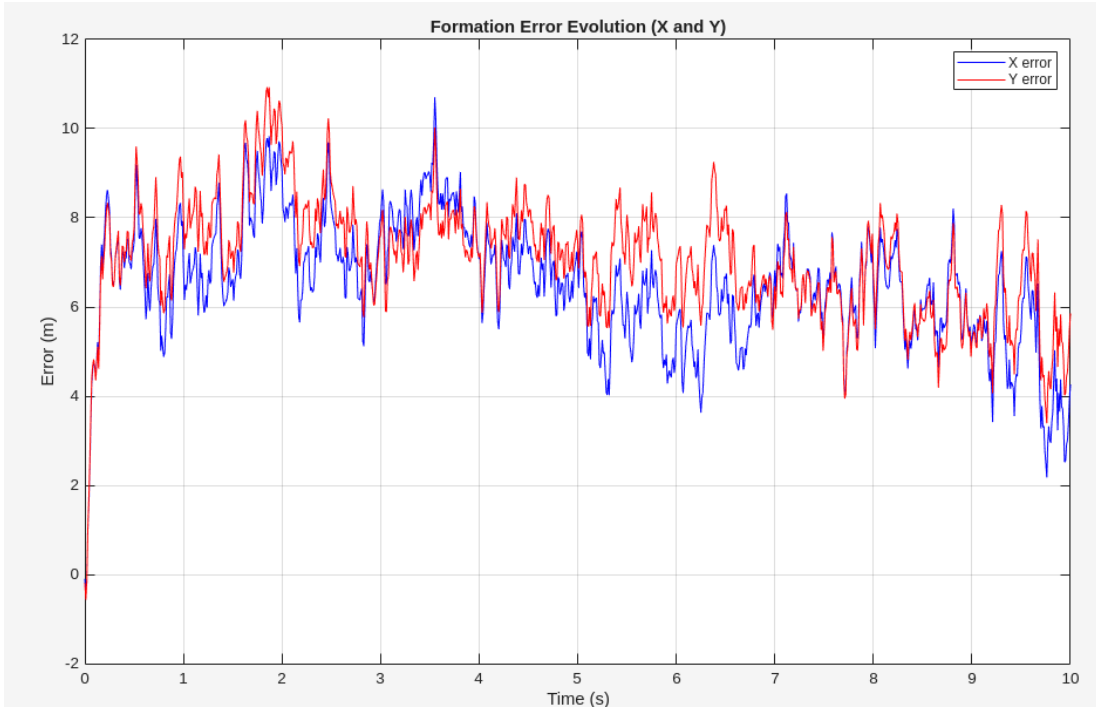

Figure 8. Error Evolution over Time

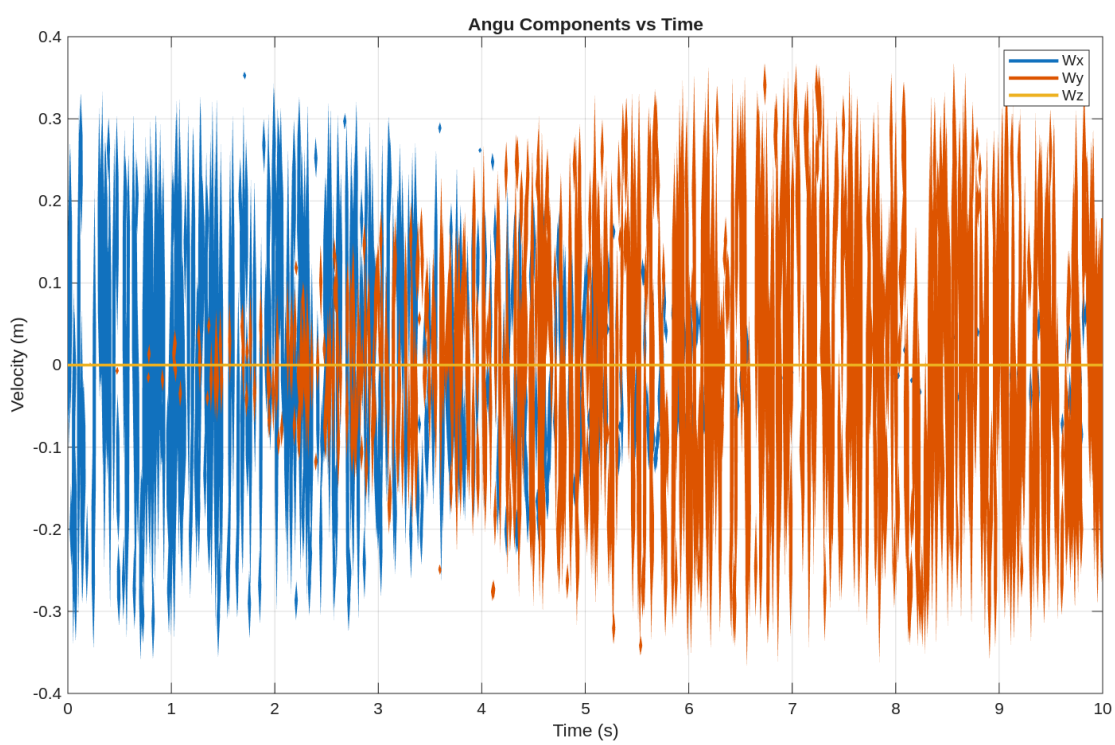

Figure 9. Velocity over Time